

Computer Organization and Assembly Language

Lecture 5

Assembly Language Fundamentals

What's Next

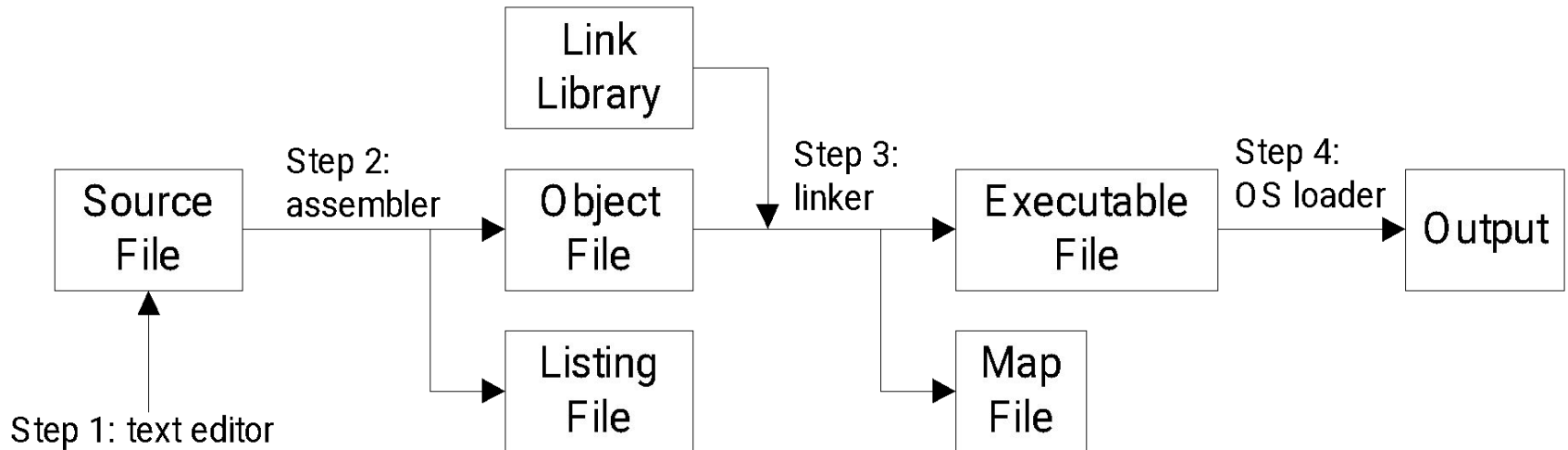
- Basic Elements of Assembly Language
 - Example: Adding and Subtracting Integers
 - **Assembling, Linking, and Running Programs**
 - Defining Data
 - Symbolic Constants
 - Real-Address Mode Programming
-

Assembling, Linking, and Running Programs

- Assemble-Link-Execute Cycle
- Listing File
- Map File

Assemble-Link Execute Cycle

- Assembly language program must be translated to machine language for the target processor.
- The following diagram describes the steps from creating a source program through executing the compiled program.
- If the source code is modified, Steps 2 through 4 must be repeated.



Assemble-Link-Execute

Step 1: A programmer uses a text editor to create an ASCII text file named the source file.

Step 2: The assembler reads the source file and produces an object file, a machine-language translation of the program. Optionally, it produces a listing file. If any errors occur, the programmer must return to Step 1 and fix the program.

Step 3: The linker reads the object file and checks to see if the program contains any calls to procedures in a link library. The linker copies any required procedures from the link library, combines them with the object file, and produces the executable file.

Step 4: The operating system loader utility reads the executable file into memory and branches the CPU to the program's starting address, and the program begins to execute. See the topic "Getting Started" on the author's Web site (www.asmirvine.com) for detailed instructions on assembling, linking, and running assembly language programs using Microsoft Visual Studio.

Listing File

- Use it to see how your program is compiled
 - Contains
 - source code
 - addresses
 - object code (machine language)
 - segment names
 - symbols (variables, procedures, and constants)
-

Map File

- Information about each program segment:
 - starting address
 - ending address
 - size
 - segment type

What's Next

- Basic Elements of Assembly Language
 - Example: Adding and Subtracting Integers
 - Assembling, Linking, and Running Programs
 - **Defining Data**
 - Symbolic Constants
 - Real-Address Mode Programming
-

Defining Data

- Intrinsic Data Types
 - Data Definition Statement
 - Defining BYTE and SBYTE Data
 - Defining WORD and SWORD Data
 - Defining DWORD and SDWORD Data
 - Defining QWORD Data
 - Defining TBYTE Data
 - Little Endian Order
 - Declaring Uninitialized Data
-

Intrinsic Data Types (1 of 2)

- BYTE, SBYTE
 - 8-bit unsigned integer; 8-bit signed integer
- WORD, SWORD
 - 16-bit unsigned & signed integer
- DWORD, SDWORD
 - 32-bit unsigned & signed integer
- QWORD
 - 64-bit integer
- TBYTE
 - 80-bit integer

Data Definition Statement

- A data definition statement sets aside storage in memory for a variable.
- May optionally assign a name (label) to the data
- Syntax:

[name] directive initializer [,initializer] . . .

value1 BYTE 10

- All initializers become binary data in memory

Defining Byte Arrays

Examples that use
multiple initializers:

```
list1 BYTE 10,20,30,40
```

```
list2 BYTE 10,20,30,40
```

```
        BYTE 50,60,70,80
```

```
        BYTE 81,82,83,84
```

```
list3 BYTE ?,32,41h,00100010b
```

```
list4 BYTE 0Ah,20h,'A',22h
```

	Offset	Value
list1	0000	10
	0001	20
	0002	30
	0003	40
list2	0004	10
	0005	20
	0006	30
	0007	40
	0008	50
	0009	60
	000A	70
	000B	80
	000C	81
	000D	82
	000E	83
	000F	84
list3	0010	

Defining Strings (1 of 3)

- A string is implemented as an array of characters
 - For convenience, it is usually enclosed in quotation marks
 - It often will be null-terminated (ending with ,0)
- Examples:

```
str1 BYTE "Enter your name",0
str2 BYTE 'Error: halting program',0
str3 BYTE 'A','E','I','O','U'
greeting BYTE "Welcome to the Encryption Demo program "
          BYTE "created by Kip Irvine.",0
```

Defining Strings (2 of 3)

- To continue a single string across multiple lines, end each line with a comma:

```
menu BYTE "Checking Account",0dh,0ah,0dh,0ah,  
        "1. Create a new account",0dh,0ah,  
        "2. Open an existing account",0dh,0ah,  
        "3. Credit the account",0dh,0ah,  
        "4. Debit the account",0dh,0ah,  
        "5. Exit",0ah,0ah,  
        "Choice> ",0
```

Defining Strings (3 of 3)

- End-of-line character sequence:
 - 0Dh = carriage return
 - 0Ah = line feed

```
str1 BYTE "Enter your name:      ",0Dh,0Ah  
      BYTE "Enter your address: ",0
```

```
newLine BYTE 0Dh,0Ah,0
```

Idea: Define all strings used by your program in the same area of the data segment.

Using the DUP Operator

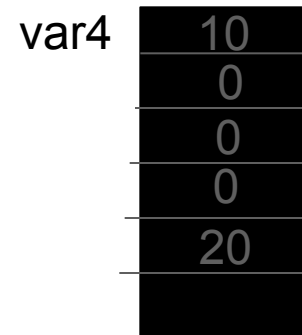
- Use DUP to allocate (create space for) an array or string. Syntax: *counter* DUP (*argument*)
- *Counter* and *argument* must be constants or constant expressions

```
var1 BYTE 20 DUP(0)      ; 20 bytes, all equal to zero
```

```
var2 BYTE 20 DUP(?)      ; 20 bytes, uninitialized
```

```
var3 BYTE 4 DUP("STACK") ; 20 bytes: "STACKSTACKSTACKSTACK"
```

```
var4 BYTE 10,3 DUP(0),20 ; 5 bytes
```



Defining WORD and SWORD Data

- Define storage for 16-bit integers
 - or double characters
 - single value or multiple values

```
word1  WORD  65535  ; largest unsigned value
```

```
word2  SWORD -32768 ; smallest signed value
```

```
word3  WORD  ?    ; uninitialized, unsigned
```

```
word4  WORD  "AB"  ; double characters
```

```
myList WORD  1,2,3,4,5 ; array of words
```

```
array  WORD  5 DUP(?) ; uninitialized array
```

Defining QWORD, TBYTE

Storage definitions for quadwords, tenbyte values,
and real numbers:

```
quad1 QWORD 1234567812345678h  
val1  TBYTE 1000000000123456789Ah
```

Little Endian Order

- All data types larger than a byte store their individual bytes in reverse order. The least significant byte occurs at the first (lowest) memory address.

- Example:

`val1 DWORD 12345678h`

0000:	78
0001:	56
0002:	34
0003:	12

Adding Variables to AddSub

```
TITLE Add and Subtract, Version 2                                (AddSub2.asm)
; This program adds and subtracts 32-bit unsigned
; integers and stores the sum in a variable.
INCLUDE Irvine32.inc
.data
val1 DWORD 10000h
val2 DWORD 40000h
val3 DWORD 20000h
finalVal DWORD ?
.code
main PROC
    mov eax,val1    ; start with 10000h
    add eax,val2    ; add 40000h
    sub eax,val3    ; subtract 20000h
    mov finalVal,eax ; store the result (30000h)
    call DumpRegs   ; display the registers
    exit
main ENDP
END main
```

Declaring Uninitialized Data

- Use the `.data?` directive to declare an uninitialized data segment:
`.data?`
- Within the segment, declare variables with "?" initializers:

```
smallArray DWORD 10 DUP(?)
```

Advantage: the program's EXE file size is reduced.

What's Next

- Basic Elements of Assembly Language
 - Example: Adding and Subtracting Integers
 - Assembling, Linking, and Running Programs
 - Defining Data
 - **Symbolic Constants**
 - Real-Address Mode Programming
-

Symbolic Constants

- Equal-Sign Directive
 - Calculating the Sizes of Arrays and Strings
 - EQU Directive
 - TEXTEQU Directive
-

Equal-Sign Directive

- *name = expression*
 - expression is a 32-bit integer (expression or constant)
 - may be redefined
 - *name* is called a **symbolic constant**
- good programming style to use symbols

```
COUNT = 500
```

```
.
```

```
.
```

```
mov ax,COUNT
```

Calculating the Size of a Byte Array

- current location counter: \$
 - subtract address of list
 - difference is the number of bytes

```
list BYTE 10,20,30,40  
ListSize = ($ - list)
```

Calculating the Size of a Word Array

Divide total number of bytes by 2 (the size of a word)

```
list WORD 1000h,2000h,3000h,4000h  
ListSize = ($ - list) / 2
```

Calculating the Size of a Doubleword Array

Divide total number of bytes by 4 (the size of a doubleword)

```
list DWORD 1,2,3,4  
ListSize = ($ - list) / 4
```

EQU Directive

- Define a symbol as either an integer or text expression.
- Cannot be redefined

```
PI EQU <3.1416>
```

```
pressKey EQU <"Press any key to continue...",0>
```

```
.data
```

```
prompt BYTE pressKey
```

TEXTEQU Directive

- Define a symbol as either an integer or text expression.
- Called a **text macro**
- Can be redefined

```
continueMsg TEXTEQU <"Do you wish to continue (Y/N)?">
rowSize = 5
.data
prompt1 BYTE continueMsg
count TEXTEQU %(rowSize * 2)      ; evaluates the expression
setupAL TEXTEQU <mov al,count>

.code
setupAL      ; generates: "mov al,10"
```

What's Next

- Basic Elements of Assembly Language
 - Example: Adding and Subtracting Integers
 - Assembling, Linking, and Running Programs
 - Defining Data
 - Symbolic Constants
 - **Real-Address Mode Programming**
-

Real-Address Mode Programming (1 of 2)

- Generate 16-bit MS-DOS Programs
- Advantages
 - enables calling of MS-DOS and BIOS functions
 - no memory access restrictions
- Disadvantages
 - must be aware of both segments and offsets
 - cannot call Win32 functions (Windows 95 onward)
 - limited to 640K program memory

Real-Address Mode Programming (2 of 2)

- Requirements
 - INCLUDE Irvine16.inc
 - Initialize DS to the data segment:

```
mov ax, @data  
mov ds, ax
```


Add and Subtract, 16-Bit Version

```
TITLE Add and Subtract, Version 2           (AddSub2r.asm)
INCLUDE Irvine16.inc
.data
val1 DWORD 10000h
val2 DWORD 40000h
val3 DWORD 20000h
finalVal DWORD ?
.code
main PROC
    mov ax,@data    ; initialize DS
    mov ds,ax
    mov eax,val1    ; get first value
    add eax,val2    ; add second value
    sub eax,val3    ; subtract third value
    mov finalVal,eax ; store the result
    call DumpRegs   ; display registers
    exit
main ENDP
END main
```

Summary

- Integer expression, character constant
 - directive – interpreted by the assembler
 - instruction – executes at runtime
 - code, data, and stack segments
 - source, listing, object, map, executable files
 - Data definition directives:
 - BYTE, WORD, DWORD, QWORD, TBYTE
 - DUP operator, location counter (\$)
 - Symbolic constant
 - EQU and TEXTEQU
-